# Zephyr RTOS: una aplicación, muchas placas

## Devicetree, drivers estandarizados y portabilidad real

Fabrizio Carlassara
Laboratorio de Sistemas Embebidos

Congreso de Ingeniería Electrónica 2026

UTN – Facultad Regional Avellaneda

# Hoja de ruta

1. El problema: código atado al fabricante
2. Zephyr en cinco minutos
3. **Devicetree**: describir el hardware en vez de programarlo
4. **Modelo de drivers**: una API, muchas implementaciones
5. Demo: las mismas aplicaciones en placas de distintos fabricantes y arquitecturas
6. Herramientas, límites y cómo empezar

# El problema

# Prender un LED, cuatro fabricantes

**STM32 (STM32Cube HAL)**

```
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13,
                  GPIO_PIN_SET);
```

**NXP (MCUXpresso SDK)**

```
GPIO_PinWrite(GPIO, 1, 4, 1U);
```

**Espressif (ESP-IDF)**

```
gpio_set_level(GPIO_NUM_2, 1);
```

**Raspberry Pi (Pico SDK)**

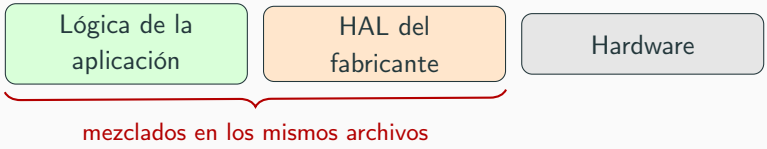
```
gpio_put(25, true);
```

Cuatro APIs, cuatro sistemas de build, cuatro formas de configurar relojes y pines.

El pin y el periférico quedan *escritos en el código*.

# El costo de casarse con un fabricante

- **Faltantes de componentes** (2021–2023): rediseñar la placa *y* reescribir el firmware
- **Cambio de requerimientos**: más RAM, BLE, menor consumo… $\Rightarrow$ otro micro
- **Equipos y formación**: cada familia es un mundo nuevo
- **Prototipo vs. producto**: se prueba en una placa de desarrollo y se fabrica con otra



¿Y si la aplicación
*no supiera* en qué placa corre?

# Zephyr en cinco minutos

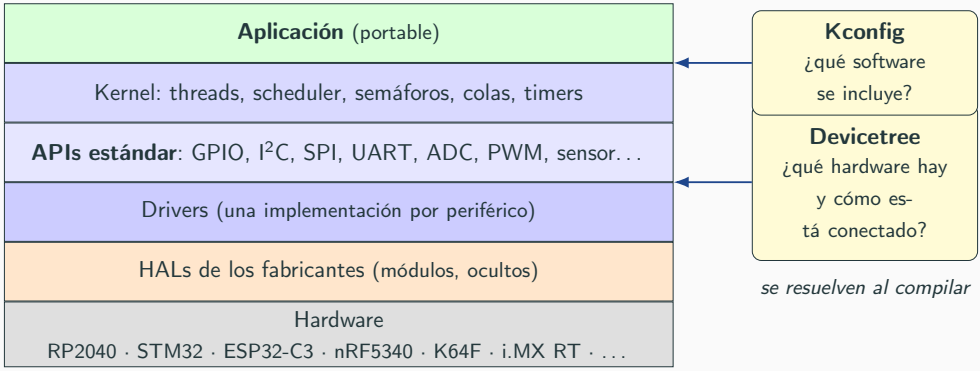
# ¿Qué es Zephyr?

- RTOS de código abierto, proyecto de la **Linux Foundation** (2016)
- Licencia **Apache 2.0**: sin regalías, apto para productos comerciales
- Gobernanza abierta: Intel, Nordic, NXP, ST, Espressif, Google, Meta...
- No es solo un kernel: **drivers, pilas de comunicación, sistema de build y herramientas**
- Versiones cada 6 meses, LTS con soporte extendido

| **Zephyr v4.5.0** | |
|---|---|
| +1100 | placas |
| 185 | fabricantes de placas |
| 13 | arquitecturas |
| +240 | drivers de sensores |

Contado sobre el árbol de v4.5.0

# Arquitectura: dónde vive cada cosa



# Zephyr vs. un RTOS "solo kernel"

| | **Kernel solo** (p. ej. FreeRTOS) | **Zephyr** |
|---|---|---|
| Scheduler, colas, semáforos | Sí | Sí |
| Drivers de periféricos | Los del fabricante (HAL propio) | Propios, con API común |
| Descripción del hardware | En el código (`#define`, init) | Devicetree |
| Configuración | `FreeRTOSConfig.h` | Kconfig (`prj.conf`) |
| Build y flasheo | El del IDE del fabricante | `west` + CMake, igual para todos |
| Red, BLE, USB, FS | Bibliotecas externas | Integradas |

No es mejor ni peor: **es otro alcance**. Zephyr ocupa más memoria y exige más formación.

# Devicetree

# Devicetree: describir, no programar

- Estructura de datos **jerárquica** que describe el hardware: buses, periféricos, pines, direcciones
- Viene de Open Firmware y del kernel Linux (ARM, 2011)
- En Linux se interpreta *al arrancar*; en Zephyr se resuelve al compilar
  - No hay parser en el microcontrolador
  - Todo termina en **macros de C** $\Rightarrow$ costo cero en tiempo de ejecución
  - Los errores (pin inexistente, propiedad faltante) aparecen **al compilar**

# Anatomía de un nodo

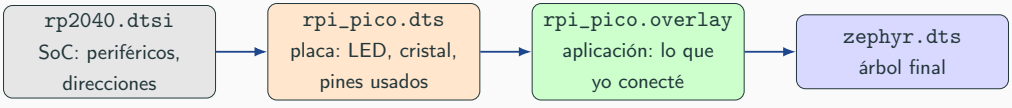
```
/ {
    aliases {
        accel0 = &mpu6050;
    };
};

&i2c1 {
    status = "okay";
    clock-frequency = <400000>;

    mpu6050: mpu6050@68 {
        compatible = "invensense,mpu6050";
        reg = <0x68>;
    };
};
```

| | |
|---|---|
| `/` | nodo raíz |
| `&i2c1` | referencia a un nodo ya definido (por su *label*) |
| `mpu6050:` | *label*: nombre para referenciarlo |
| `@68` | *unit address*: dirección en el bus |
| `compatible` | qué es el dispositivo ⇒ qué driver lo maneja |
| `reg` | dirección $I^2C$ |
| `status` | habilitado o no |
| `aliases` | nombres genéricos que usa la aplicación |

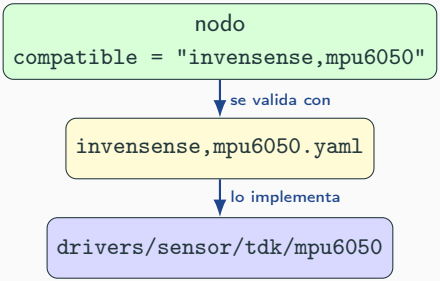
# Capas: del silicio a la aplicación



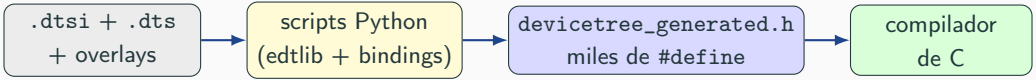

Cada capa **agrega o modifica** nodos de la anterior. La aplicación nunca toca las dos primeras.

# Bindings: el contrato entre el árbol y el driver

- Archivo YAML por cada `compatible`
- Define qué propiedades existen, su tipo y si son obligatorias
- El build **valida** el árbol contra el binding
- Zephyr v4.5.0 trae más de 4400 bindings



# Del árbol al código C




```c
/* La aplicación pide "lo que sea que esté en led0"... */
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

/* ...y el preprocesador lo resuelve, por ejemplo en la Pico, a:
 *   { .port = &gpio0_device, .pin = 25, .dt_flags = GPIO_ACTIVE_HIGH }
 * Sin tablas en RAM, sin búsquedas en tiempo de ejecución. */
```

# Kconfig: la otra mitad de la configuración

**prj.conf de la app del acelerómetro**

```
# APIs que usa la aplicación
CONFIG_GPIO=y
CONFIG_I2C=y
CONFIG_SENSOR=y

# Logging (con punto flotante)
CONFIG_LOG=y
CONFIG_CBPRINTF_FP_SUPPORT=y

# Shell para explorar el hardware
CONFIG_SHELL=y
CONFIG_I2C_SHELL=y
CONFIG_SENSOR_SHELL=y
```

- **Devicetree**: qué hardware hay
- **Kconfig**: qué software se compila
- Los drivers se activan solos: si hay un nodo `"invensense,mpu6050"` con `status = "okay"`, se compila su driver
- Un mismo `prj.conf` para todas las placas
- `west build -t menuconfig` para explorar

# Modelo de drivers

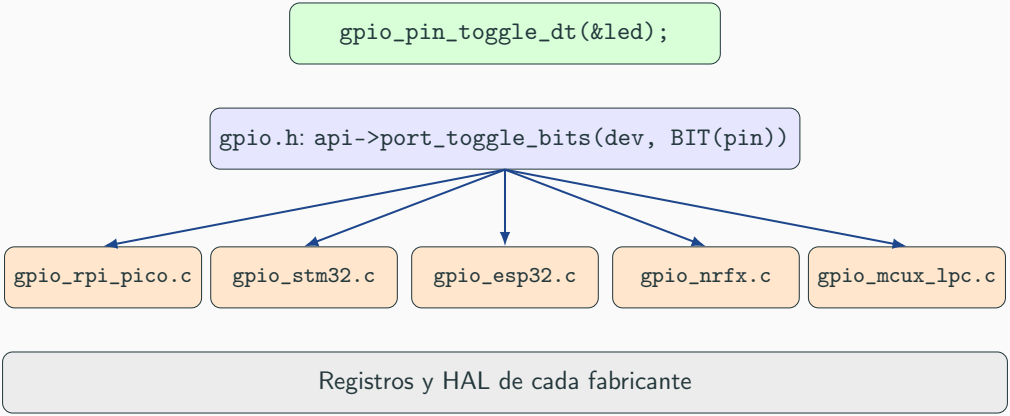
# `struct device`: todo dispositivo es igual

```
struct device {
    const char *name;
    const void *config; /* flash */
    void *data;         /* RAM   */
    const void *api;    /* vtable */
    /* ... */
};
```

**config**: datos del devicetree (constantes)
**data**: estado del driver
**api**: tabla de funciones del driver

- **Una instancia por nodo** del devicetree con `status = "okay"`
- Se crean **al compilar** (`DEVICE_DT_INST_DEFINE`)
- Se inicializan solas antes de `main()`, por niveles: `PRE_KERNEL`, `POST_KERNEL`...
- La app obtiene el puntero con `DEVICE_DT_GET(nodo)`

# Una API, muchas implementaciones




La función que se ejecuta la decide **el devicetree**, a través del puntero `api` del dispositivo.

# La API de sensores

```
const struct device *s =
        DEVICE_DT_GET(DT_ALIAS(accel0));
struct sensor_value v[3];

sensor_sample_fetch(s);  /* medir */

sensor_channel_get(s,
        SENSOR_CHAN_ACCEL_XYZ, v);  /* m/s2 */
sensor_channel_get(s,
        SENSOR_CHAN_GYRO_XYZ, v);   /* rad/s */
/* canal que no mide -> -ENOTSUP */
```

- Canales con **unidades del SI**: $m/s^2$, rad/s, gauss, °C
- `sensor_value`: entero + millonésimas, sin `float` obligatorio
- Igual para MPU6050, FXOS8700, FXLS8974, MMA8652FC y otros +240
- API asíncrona `sensor_read()` + decodificadores para alto rendimiento

# APIs estandarizadas: no solo GPIO

| **Periféricos** | **Comunicación** | **Servicios** |
|---|---|---|
| GPIO, PWM, ADC, DAC | UART, $I^2C$, SPI, CAN | Logging, shell |
| Contadores, watchdog, RTC | USB device/host | Sistema de archivos |
| Sensores, LED, displays | Bluetooth LE | Settings (NVS) |
| Entrada (teclas, touch) | Ethernet, Wi-Fi, TCP/IP | Gestión de energía |
| Flash, EEPROM | Thread, Matter, LoRa | Actualización (MCUboot) |

Mismo patrón en todas: `DEVICE_DT_GET` + `device_is_ready` + funciones de la API.

# Demo

# El banco de pruebas: solo hardware integrado

| Placa | Fabricante | Núcleo | LED | Botón | Acelerómetro | Temp. |
|---|---|---|---|---|---|---|
| nRF5340 DK | Nordic | Cortex-M33 | ✓ | ✓ | MPU6050 (ext.) | ✗ |
| FRDM-K64F | NXP | Cortex-M4F | ✓ | ✓ | FXOS8700 | ✓ |
| LPCXpresso55S69 | NXP | Cortex-M33 | ✓ | ✓ | MMA8652FC | ✗ |
| FRDM-MCXN236 | NXP | Cortex-M33 | ✓ | ✓ | FXLS8974 | ✓ |
| ESP32-C3 SuperMini | Espressif | RISC-V | ✓ | ✓ | ✗ | ✓ |
| DOIT ESP32 | Espressif | Xtensa LX6 | ✓ | ✓ | ✗ | ✗ |
| Teensy 4.1 | PJRC (NXP) | Cortex-M7 | ✓ | ✗ | ✗ | ✓ |

Cada demo usa lo que la placa ya trae. Única excepción: el MPU6050 del nRF5340 DK.
También compilan, como referencia de memoria: Pico, Pico 2, Black Pill F401 y ESP32-S3.

# App 1: `blinky_boton` — el `main.c` (lo esencial)

```c
static const struct gpio_dt_spec led   = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
static const struct gpio_dt_spec boton = GPIO_DT_SPEC_GET(DT_ALIAS(sw0),  gpios);

int main(void)
{
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&boton, GPIO_INPUT);
    gpio_pin_interrupt_configure_dt(&boton, GPIO_INT_EDGE_TO_ACTIVE);
    gpio_init_callback(&boton_cb, boton_presionado, BIT(boton.pin));
    gpio_add_callback_dt(&boton, &boton_cb);

    while (1) {
        gpio_pin_toggle_dt(&led);
        k_msleep(periodos_ms[atomic_get(&indice)]);
    }
}
```

Ni un número de pin. Ni un registro. Ni un `#ifdef` de placa (solo "¿hay botón?").

# Lo que cambia por placa: nada

**El botón es opcional**

```
#if DT_NODE_EXISTS(DT_ALIAS(sw0))
/* la placa tiene botón: se usa */
static const struct gpio_dt_spec boton =
    GPIO_DT_SPEC_GET(DT_ALIAS(sw0), gpios);
#else
/* sin botón: periodo fijo */
#endif
```

La Teensy 4.1 no tiene botón: la app lo sabe **al compilar**.

**Ningún overlay para blinky**

- Todas las placas de la demo ya definen `led0` (y casi todas `sw0`) en su `.dts`

```
west build -p -b nrf5340dk/nrf5340/cpuapp \
    apps/blinky_boton
west build -p -b esp32c3_supermini \
    apps/blinky_boton
west build -p -b teensy41 apps/blinky_boton
```

# App 2: acelerometro — no sabe qué sensor tiene

```c
/* "accel0": el nombre estándar de Zephyr para el acelerómetro de la placa */
static const struct device *const sensor = DEVICE_DT_GET(DT_ALIAS(accel0));

static const struct magnitud magnitudes[] = {
    {SENSOR_CHAN_ACCEL_XYZ,    3, "Aceleracion", "m/s2"},
    {SENSOR_CHAN_GYRO_XYZ,     3, "Giro",        "rad/s"},
    {SENSOR_CHAN_MAGN_XYZ,     3, "Campo magn.", "G"},
    {SENSOR_CHAN_DIE_TEMP,     1, "Temp. chip",  "C"},
    {SENSOR_CHAN_AMBIENT_TEMP, 1, "Temperatura", "C"},
};

while (1) {
    sensor_sample_fetch(sensor);
    for (size_t i = 0; i < ARRAY_SIZE(magnitudes); i++) {
        mostrar(&magnitudes[i]);  /* si el driver responde -ENOTSUP, se omite */
    }
    k_msleep(2000);
}
```

# Placa + sensor: overlays que se combinan

**frdm_k64f.dts** (de Zephyr, no lo tocamos)

```
aliases { accel0 = &fxos8700; };
```

**boards/frdm_k64f.overlay** (se aplica solo)

```
sensor_i2c: &i2c0 {   /* mi bus I2C */
    status = "okay";
};
```

**mpu6050.overlay** (se pide al compilar)

```
&sensor_i2c {
    mpu6050: mpu6050@68 {
        compatible = "invensense,mpu6050";
        reg = <0x68>;
    };
};
/ { aliases { accel0 = &mpu6050; }; };
```

```
# acelerómetro integrado: sin overlay extra
west build -p -b frdm_k64f apps/acelerometro
west build -p -b frdm_mcxn236 apps/acelerometro

# MPU6050 externo (el nRF5340 DK no trae)
west build -p -b nrf5340dk/nrf5340/cpuapp \
  apps/acelerometro \
  -- -DEXTRA_DTC_OVERLAY_FILE=mpu6050.overlay
```

El overlay extra se aplica **después** del de la placa: si la placa tenía sensor, el MPU6050 lo reemplaza.

$N$ placas + $M$ sensores = $N + M$ archivos, no $N \times M$.

# Mismo `main.c`, distinto sensor

| Sensor | Dónde | Acel. | Giro | Magn. | Temp. |
|---|---|---|---|---|---|
| MPU6050 | externo (nRF5340 DK) | ✓ | ✓ | ✗ | ✓ |
| FXOS8700 | integrado en FRDM-K64F | ✓ | ✗ | ✓ | ✗ |
| FXLS8974 | integrado en FRDM-MCXN236 | ✓ | ✗ | ✗ | ✓ |
| MMA8652FC | integrado en LPC55S69 | ✓ | ✗ | ✗ | ✗ |

- La app pide todas las magnitudes; cada driver **responde lo que mide**
- `accel0` ya lo definen las placas: la app no necesita overlay
- FXOS8700 y MMA8652FC: dos chips, un solo driver (`nxp,fxos8700`)
- El modo (solo acelerómetro o acelerómetro + magnetómetro) lo fija el **Kconfig de cada placa**

# App 3: `temperatura` — el sensor interno del chip

```c
/* "die-temp0": el nombre estándar de Zephyr para la temperatura del chip */
static const struct device *const sensor = DEVICE_DT_GET(DT_ALIAS(die_temp0));

while (1) {
    sensor_sample_fetch_chan(sensor, SENSOR_CHAN_DIE_TEMP);
    sensor_channel_get(sensor, SENSOR_CHAN_DIE_TEMP, &temp);
    LOG_INF("Temperatura del chip: %.1f C", sensor_value_to_double(&temp));
    k_msleep(2000);
}
```

Casi todos los microcontroladores tienen uno. Cada fabricante lo mide distinto.

# Mismo alias, distinto hardware

| SoC | Cómo mide | Qué hace el overlay |
|---|---|---|
| ESP32-C3, ESP32-S3 | periférico dedicado | habilitar el nodo |
| i.MX RT1062 (Teensy 4.1) | monitor dedicado (TEMPMON) | habilitar el nodo |
| RP2040 (Pico) | canal 4 del ADC | habilitar el nodo |
| RP2350 (Pico 2) | canal 4 del ADC | nada: ya viene habilitado |
| STM32F401 (Black Pill) | ADC1 canal 16 + calibración | habilitar + alias |
| Kinetis K64F | ADC1 canales 26 y 27 | solo el alias |
| MCXN236 | LPADC0 canales 26A/26B | configurar el canal |

La aplicación siempre pide lo mismo: `die-temp0` y `SENSOR_CHAN_DIE_TEMP`.
El nRF5340 tiene sensor, pero en el núcleo de red: desde el de aplicación no está disponible.

## Demo en vivo

**Tres apps, tres arquitecturas,**
**ni un `main.c` modificado**

# Resultados: lo que cuesta cada app

| Placa | Núcleo | blinky_boton | acelerometro | temperatura |
|---|---|---|---|---|
| Raspberry Pi Pico | Cortex-M0+ | 25,8 / 6,0 | 92,5 / 17,2 | 36,0 / 6,2 |
| Raspberry Pi Pico 2 | Cortex-M33 | 26,6 / 6,1 | 90,0 / 17,6 | 35,3 / 6,2 |
| Black Pill F401 | Cortex-M4F | 27,6 / 6,6 | 94,3 / 18,3 | 41,9 / 7,0 |
| DOIT ESP32 | Xtensa LX6 | 144,9 / 52,2 | 188,9 / 66,1 | – |
| ESP32-C3 SuperMini | RISC-V | 131,8 / 91,1 | 211,9 / 116,3 | 134,1 / 93,6 |
| ESP32-S3 DevKitC | Xtensa LX7 | – | 148,3 / 121,9 | 135,3 / 98,4 |
| nRF5340 DK | Cortex-M33 | 31,9 / 6,6 | 94,2 / 17,8 | – |
| LPCXpresso55S69 | Cortex-M33 | 30,1 / 6,4 | 94,5 / 17,6 | – |
| FRDM-MCXN236 | Cortex-M33 | 38,1 / 6,5 | 100,8 / 18,0 | 54,4 / 7,2 |
| FRDM-K64F | Cortex-M4F | 27,5 / 6,4 | 94,6 / 18,2 | 41,4 / 6,9 |
| Teensy 4.1 | Cortex-M7 | 42,1 / 6,5 | 104,8 / 15,3 | 51,0 / 6,5 |

KiB de flash / RAM según el enlazador (Zephyr 4.5.0). 32 compilaciones, ningún `main.c` modificado. `acelerometro` incluye la shell; "–": la placa no tiene ese hardware.

# Explorar el hardware desde la shell

```
uart:~$ device list
uart:~$ i2c scan i2c@40005400
     0  1  2  3  4  5  6  7  8  9  a  b  ...
60: -- -- -- -- -- -- -- -- 68 -- -- --
uart:~$ sensor get mpu6050@68
uart:~$ kernel thread list
```

- La shell está en el `prj.conf`: viene en la app, sin recompilar nada
- Comandos para GPIO, $I^2C$, sensores, threads, memoria
- Ideal para validar el cableado antes de escribir código

# Herramientas y límites

# `west`: una sola herramienta para todo

```
west init -l cie2026-zephyr    # workspace
west update                    # Zephyr + HALs
west build -b <placa> <app>
west flash                     # grabar
west debug                     # depurar (GDB)
west build -t menuconfig       # Kconfig
```

- `west flash` elige el programador según la placa: `pyocd`, `esptool`, `nrfutil`, `linkserver`, `teensy`, J-Link…
- El **manifest** (`west.yml`) fija versiones ⇒ builds reproducibles
- Funciona igual en Linux, macOS y Windows

# Lo que Zephyr *no* resuelve

- **Periféricos exclusivos** de un fabricante (PIO del RP2040, radio de Nordic, ULP del ESP32): portables solo en parte
- **Soporte desparejo**: no todos los drivers implementan todo (DMA, bajo consumo, *async*)
- **Costo en memoria**: más que bare-metal o un kernel solo (blinky ≈30 KiB; con logging y shell 90–105 KiB en Cortex-M)
- **Cobertura desigual**: el sensor de temperatura no está disponible en el nRF5340 (núcleo de aplicación), el ESP32 ni el LPC55S69
- **Curva de aprendizaje**: devicetree, Kconfig, CMake y west de entrada
- **Temporización y consumo** siguen dependiendo del silicio: portar no es lo mismo que validar

La portabilidad no es gratis: se paga una vez, en la arquitectura, y no en cada cambio de placa.

# Para llevarse

1. El **devicetree** separa *qué hardware hay* de *qué hace la aplicación*
2. Las **APIs estandarizadas** hacen que el driver sea un detalle de implementación
3. Cambiar de placa o de sensor = **otro overlay**, no otro `main.c`
4. El código de la aplicación se vuelve **reutilizable, testeable y enseñable**

# Probalo vos

- Workspace completo: manifest, apps y overlays
- Guía paso a paso para tu computadora (Linux, macOS o Windows)
- Nota técnica con los resultados

`https://github.com/utn-fra-lse/cie2026-zephyr`



**Laboratorio de Sistemas Embebidos**
UTN – Facultad Regional Avellaneda

¿Preguntas?